

deti
universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Process/Thread Synchronization Basics

Real-Time Embedded Systems (#41505)

Paulo Pedreiras, pbrp@ua.pt
Department of Electronics, Telecommunications and Informatics
University of Aveiro
Academic year 2026/2027

Outline

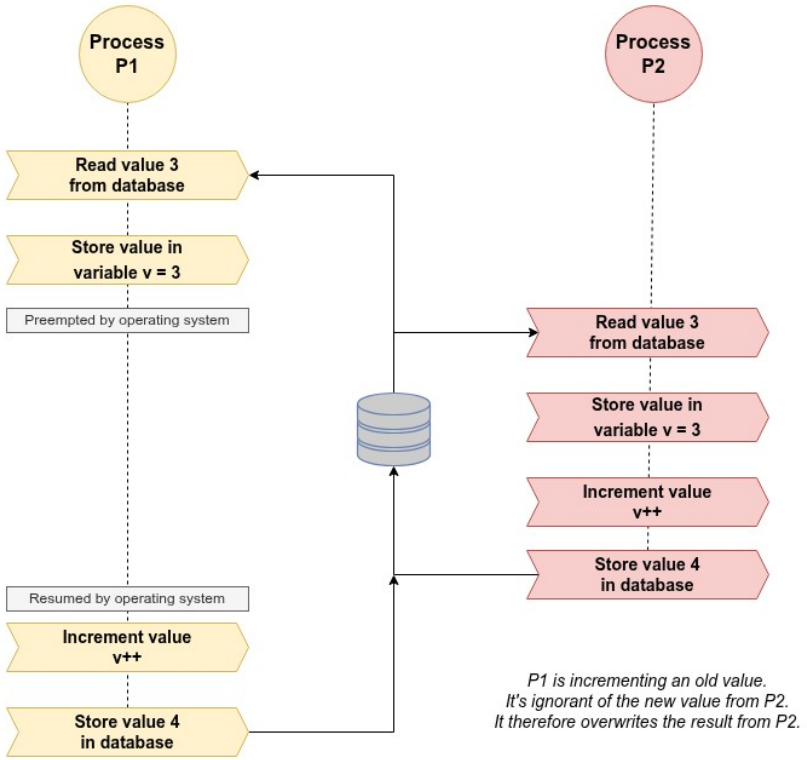
- Background
- The Critical-Section Problem
- Peterson’s Solution
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Liveness

Background

- Processes/threads/tasks can execute concurrently
  - May be interrupted at any time, partially completing execution
- **Concurrent access** to shared data may result in **data inconsistency**
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
  - This issue was introduced a couple of classes ago

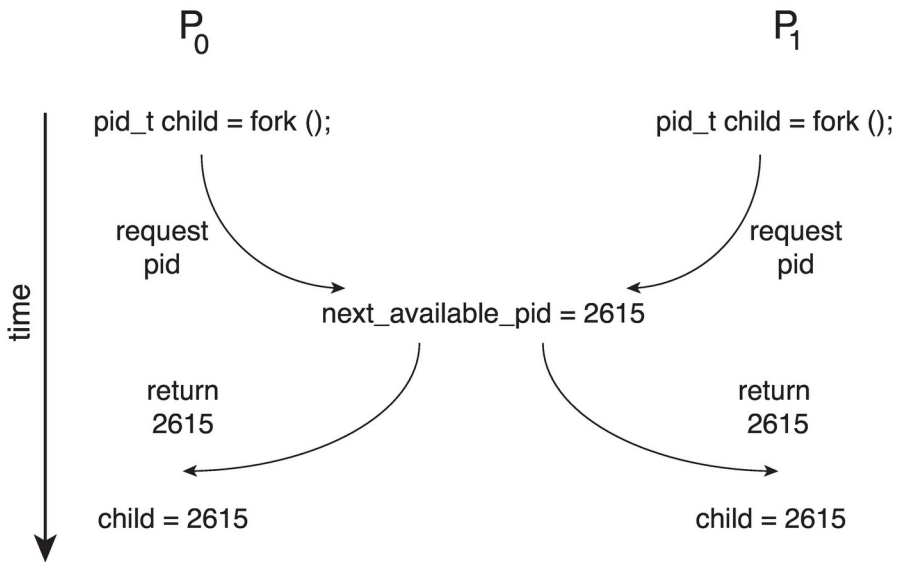
A **race condition** happens when the **behavior or result** of a program **depends** on the **timing** or **interleaving** of operations between multiple threads/processes that access shared resources

* Computation result is unpredictable! *



Race Condition - kernel level example

- Processes P_0 and P_1 are creating a child processes using the `fork()` system call
- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)

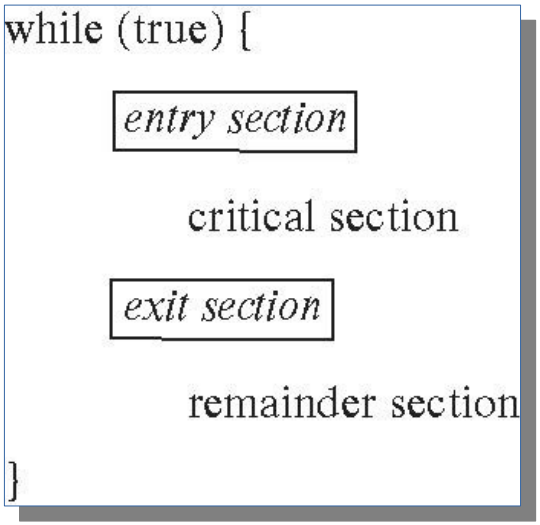


- Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable `next_available_pid` consistently the same pid could be assigned to two different processes!

Critical Section Problem

- Consider a system of ***n*** processes {p_0, p_1, … p_{n-1}}
- Each process has a **critical section** segment of code
  - Processes may be changing common variables, updating a table, writing a file, etc.
  - When one process is in a critical section, no other may be in its critical section
- *The **Critical Section Problem (CSP)*** is to design protocol to solve this
- Each process must:
  I. Ask permission to enter critical section (**entry section)**
  II. Executes critical section code
  III. Releases critical section (**exit section)**
  IV. Then executes the remaining code

General structure



Critical Section Problem (Cont.)

Requirements for solution to CSP

1. **Mutual Exclusion** - If process P_i is executing in a given critical section, then no other processes can be executing in that same critical section
2. **Progress** - If no process is executing in a given critical section and there exist one or more processes that wish to enter that critical section, then the selection of the process that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted

These requirements:

- Assume that each process executes at a nonzero speed
- No assumption concerning **relative speed** of the ***n*** processes

Interrupt-based Solution

- Possible solution (?)

  Entry section: disable interrupts

  Exit section: enable interrupts

Interrupt-based Solution

- It “**works**” on uniprocessor systems, but is **very intrusive.**
  - All processes, threads and interrupt service routines are blocked
  - Response -time of Interrupt Service Routines is often critical.
  - Use with care:
    - Very short critical regions, only if necessary e.g. for performance reasons

- But **it does not work on multiprocessor systems**
  - Disabling interrupts is a **local, core-specific hardware operation** (e.g., the cli instruction in x86).
    - It only disables interrupts on the specific processor executing the instruction.
    - All other processors in the system remain fully operational and can simultaneously access the exact same shared memory region, resulting in a race condition.

A classical SW approach - Peterson's Solution

- Two process solution
- Assume that the **`load`** and **`store`** machine-language instructions are atomic (i.e. they execute without interruption)
- The two processes share two variables:
  - **`int turn;`**
  - **`boolean flag[2]`**
- The variable **`turn`** indicates whose turn it is to enter the critical section
- The **`flag`** array is used to indicate if a process is ready to enter the critical section.
  - **`flag[i] = true`** implies that process P_i is ready!

Algorithm for Process P_i

```
while (true){

        flag[i] = true;
        turn = j;
        while (flag[j] && turn = = j)
                ;

           /* critical section */

        flag[i] = false;

         /* remainder section */

}
```

Correctness of Peterson's Solution

- The three CS requirement are met:
  1. Mutual exclusion is preserved

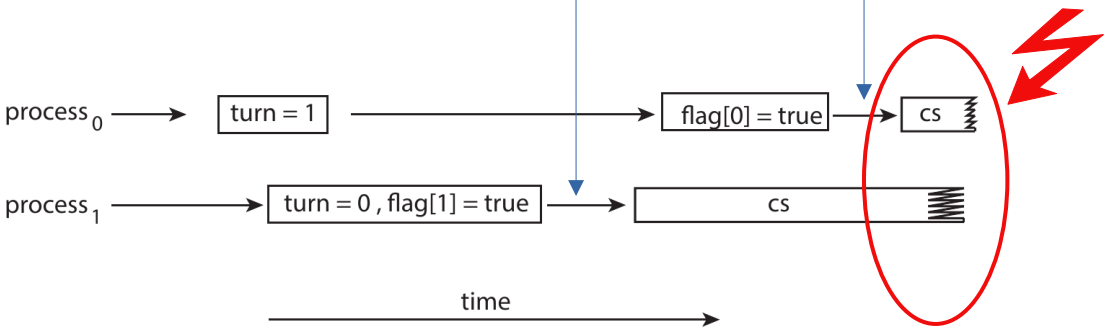
     P_i enters CS only if:

     either **`flag[j] = false`** or **`turn = i`**
  2. Progress requirement is satisfied
  3. Bounded-waiting requirement is met

Peterson's Solution and Modern Architectures

- Although useful for discussing the problem, Peterson's Solution is not guaranteed to work on modern architectures.
  - To improve performance, processors and/or compilers may reorder operations that have **no direct dependencies**
  - Understanding why it will not work is useful for better understanding race conditions.
    - For multithreaded execution the reordering may produce inconsistent or unexpected results!

```
Remember the access condition:
while (flag[j] && turn = = j;
```



Possible outcome if the first two statements that appear in the entry section of Peterson's solution are reordered

Memory Barrier

- A **Memory Model** defines the memory guarantees a computer architecture makes to application programs.
- Memory models may be either:
  - **Strongly ordered**
    - A memory modification of one processor is immediately visible to all other processors.
  - **Weakly ordered**
    - A memory modification of one processor may not be immediately visible to all other processors.
- A **memory barrier** is an instruction that forces any change in memory to be propagated (made visible) to all other processors before progress can occur.

Memory Barrier Instructions

- When a memory **barrier instruction is performed**, the system ensures that **all loads and stores are completed** before any subsequent load or store operations are performed.
- Therefore, even if instructions are reordered, a memory barrier ensures that the store operations are completed in memory and visible to other processors before future load or store operations are performed, solving the Peterson's Solution issue.
  - Anyway, SW-based solutions are inherently limited and under-performing (even with HW support such as memory barriers), so will not be further considered

Synchronization Hardware

- Many systems provide hardware support for implementing the critical section code.
- Uniprocessors – could disable interrupts
  - Currently running code would execute without preemption
    - Feasible only for very short CS
  - Generally too inefficient on multiprocessor systems (have to disable/enable ints on all processing cores)
    - Operating systems using this approach are not broadly scalable
- There are other forms of hardware support:

1. Hardware instructions
2. Atomic variables

Hardware Instructions

- Special hardware instructions that allow us to either *test-and-modify* the content of a word, or to *swap* the contents of two words atomically (uninterruptedly.)
  - **Test-and-Set** instruction
  - **Compare-and-Swap** instruction

The test_and_set Instruction

- Definition

```
    boolean test_and_set (boolean
*target)
      {
            boolean rv = *target;
            *target = true;
            return rv:
      }
```

- Properties
  - Executed atomically
  - Returns the original value of passed parameter
  - Set the new value of passed parameter to **`true`**

Solution Using test_and_set()

- Shared boolean variable **`lock`**, initialized to **`false`**
- Solution:

```
do {
        while (test_and_set(&lock))
        ; /* do nothing */

            /* critical section */

    lock = false;
            /* remainder section */
 } while (true);
```

- Does it solve the critical-section problem?

The compare_and_swap Instruction

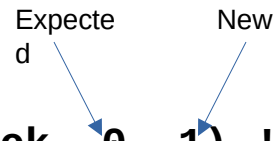
- Definition

```
int compare_and_swap(int *value, int expected, int new_value)
 {
   int temp = *value;
   if (*value == expected)
       *value = new_value;
    return temp;
 }
```

- Properties
  - Executed atomically
  - Returns the original value of passed parameter **`value`**
  - Set the variable **`value`** the value of the passed parameter **`new_value`** but only if **`*value == expected`** is true. That is, the swap takes place only under this condition.

Solution using compare_and_swap

- Shared integer **`lock`** initialized to 0;
- Solution:




```
while (true){
        while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */

        /* critical section */

        lock = 0;

    /* remainder section */
}
```

- Does it solve the critical-section problem?

Mutex Locks

- Previous solutions (and there are more, e.g. atomic variables) are complicated and generally inaccessible to application programmers
- Most OSs provide high-level tools to solve CSP in a convenient way
- Simplest is **mutex** lock
  - Boolean variable indicating if lock is available or not
- Protect a critical section by
  - First **`acquire()`** a lock
  - Then **`release()`** the lock
- Calls to **`acquire()`** and **`release()`** must be **atomic**
  - Usually implemented via hardware atomic instructions such as compare-and-swap.
- But this solution requires **busy waiting**
  - This lock is therefore called a **spinlock**

Solution to CS Problem Using Mutex Locks

```
while (true) {
        acquire lock

            critical section

        release lock

remainder section
}
```

Semaphores

- Synchronization tool that provides more sophisticated ways (than Mutex locks) for processes to synchronize their activities.
- Semaphore ***S*** – integer variable
- Can only be accessed via two indivisible (atomic) operations
  - **`wait()`** and **`signal()`**
    - Originally called **`P()`** and **`V()`**
- Definition of the **`wait()` operation**

```
wait(S) {
    while (S <= 0)
       ; // busy wait
    S--;
}
```

- Definition of the **`signal()` operation**

```
signal(S) {
    S++;
}
```

Semaphore (Cont.)

- **Counting semaphore** – integer value can range over an unrestricted domain
- **Binary semaphore** – integer value can range only between 0 and 1
  - **Although binary semaphores are sometimes described as being "the same as a mutex," this is not entirely accurate:**
    - Mutexes include **ownership** — only the task that locks the mutex can unlock it.
    - **Semaphores** have **no ownership** — any task can give (signal) or take (wait).
  - Because mutexes track ownership, they support priority-inheritance-based protocols such as PiP (Priority Inheritance Protocol), PCP (Priority Ceiling Protocol), and SRP (Stack Resource Policy), which are **essential in real-time systems.**
  - In many RTOSes, a mutex with priority inheritance is implemented internally as a binary semaphore with additional bookkeeping for priority and ownership tracking.
- With semaphores we can solve various synchronization problems

Semaphore Usage Example

- Solution to the CS Problem
  - Create a semaphore "**mutex**" initialized to 1

```
wait(mutex);
    CS
signal(mutex);
```

- Consider P_1 and P_2 that with two statements S_1 and S_2 and the requirement that S_1 to happen before S_2
  - Create a semaphore "**synch**" initialized to 0

```
P1:
    S1;
    signal(synch);
P2:
    wait(synch);
    S2;
```

Semaphore Implementation

- Must guarantee that no two processes can execute the **`wait()`** and **`signal()`** on the same semaphore at the same time
- Thus, the implementation becomes the critical section problem where the **`wait`** and **`signal`** code are placed in the critical section
- Could now have **busy waiting** in critical section implementation
  - But implementation code is short
  - Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution

Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - Value (of type integer)
  - Pointer to next record in the list
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue

Implementation with no Busy waiting (Cont.)

- Waiting queue

```
typedef struct {
        int value;
        struct process *list;
} semaphore;
```

Implementation with no Busy waiting (Cont.)

```
wait(semaphore *S) {
   S->value--;
   if (S->value < 0) {
      add this process to S->list;
      block();
   }
}

signal(semaphore *S) {
   S->value++;
   if (S->value <= 0) {
      remove a process P from S->list;
      wakeup(P);
   }
}
```

Problems with Semaphores

- Incorrect use of semaphore operations:
  - **`signal(mutex) …. wait(mutex)`**
  - **`wait(mutex) … wait(mutex)`**
  - Omitting of **`wait (mutex)`** and/or **`signal (mutex)`**
- These – and others – are examples of what can occur when semaphores and other synchronization tools are used incorrectly.

Condition Variables

- **`condition x, y;`**
- Two operations are allowed on a condition variable:
  - **`x.wait()`** – a process that invokes the operation is suspended until **`x.signal()`**
  - **`x.signal()`** – resumes one of processes (if any) that invoked **`x.wait()`**
    - If no **`x.wait()`** on the variable, then it has no effect on the variable

Usage of Condition Variable - Example

- Consider P_1 and P_2 that that need to execute two statements S_1 and S_2 and the requirement that S_1 to happen before S_2
  - Create a monitor with two procedures F_1 and F_2 that are invoked by P_1 and P_2 respectively
  - One condition variable “x” initialized to 0
  - One Boolean variable “done”
  - **F1:**

```
S1;
done = true;
x.signal();
```

  - **F2:**

```
if done = false
   x.wait()
S2;
```

Implementation – Condition Variables

- For each condition variable ***x***, we have:

```
semaphore x_sem; // (initially  = 0)
int x_count = 0;
```

- The operation **x.wait()** can be implemented as:

```
x_count++;
if (next_count > 0)
        signal(next);
else
       signal(mutex);
wait(x_sem);
x_count--;
```

Note: **next_count** counts how many threads have been signaled and are waiting to re-acquire the monitor mutex
x_count counts the number of threads waiting on that condition

Implementation (Cont.)

- The operation **`x.signal()`** can be implemented as:

```
if (x_count > 0) {
        next_count++;
        signal(x_sem);
        wait(next);
        next_count--;
}
```

Liveness

- Processes may have to wait indefinitely while trying to acquire a synchronization tool such as a mutex lock or semaphore.
- Waiting indefinitely violates the progress and bounded-waiting criteria discussed at the beginning of this chapter.
- **Liveness** refers to a set of properties that a system must satisfy to ensure processes make progress.
- Indefinite waiting is an example of a liveness failure.

Liveness

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let ***S*** and ***Q*** be two semaphores initialized to 1

P_0	P_1
`wait(S);`	`wait(Q);`
`wait(Q);`	`wait(S);`
`...`	`...`
`signal(S);`	`signal(Q);`
`signal(Q);`	`signal(S);`

- Consider if P_0 executes wait(S) and P_1 wait(Q). When P_0 executes wait(Q), it must wait until P_1 executes signal(Q)
- However, P_1 is waiting until P_0 execute signal(S).
- Since these signal() operations will never be executed, P_0 and P_1 are **deadlocked**.

Liveness

- Other forms of deadlock:

  Starvation – indefinite blocking

  - A process may never be removed from the semaphore queue in which it is suspended

  Priority Inversion – Scheduling problem when lower-priority processes execute while a higher-priority process waits on a mutex

  - Always exists – by definition of CS, the process that tries to get the lock must wait for the one that it is there
  - But can duration can be be long and has to predict

    Protocols such as Priority-Inheritance Protocol, Priority Ceiling Protocol, Stack Resource Policy and others **bound the maximum duration of the priority inversion sections**

    Some also prevent deadlocks

    Topic addressed in the Real-Time part of the course

Linux Synchronization

- Linux:
  - Prior to kernel Version 2.6, disables interrupts to implement short critical sections
  - Version 2.6 and later, fully preemptive
- Linux provides:
  - Semaphores
  - Atomic integers
  - Spinlocks
  - Reader-writer versions of both
- On single-CPU system, spinlocks replaced by enabling and disabling kernel preemption

Linux Synchronization

- Atomic variables

  `atomic_t` is the type for atomic integer
- Consider the variables

```
atomic_t counter;
int value;
```

Atomic Operation	*Effect*
`atomic_set(&counter,5);`	`counter = 5`
`atomic_add(10,&counter);`	`counter = counter + 10`
`atomic_sub(4,&counter);`	`counter = counter - 4`
`atomic_inc(&counter);`	`counter = counter + 1`
`value = atomic_read(&counter);`	`value = 12`

POSIX Synchronization

- POSIX API provides
  - mutex locks
  - semaphores
  - condition variable
- Widely used on UNIX, Linux, and macOS

POSIX Mutex Locks

- Creating and initializing the lock

```
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex,NULL);
```

- Acquiring and releasing the lock

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

POSIX Semaphores

- POSIX provides two versions – **named** and **unnamed**.
- Named semaphores can be used by unrelated processes, unnamed cannot.

POSIX Named Semaphores

- Creating an initializing the semaphore:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

- Another process can access the semaphore by referring to its name **SEM**.
- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```

POSIX Unnamed Semaphores

- Creating an initializing the semaphore:

```
#include <semaphore.h>
sem_t sem;

/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```

POSIX Condition Variables

- Since POSIX is typically used in C/C++ and these languages do not provide a monitor, POSIX condition variables are associated with a POSIX mutex lock to provide mutual exclusion
- Creating and initializing the condition variable:

```
pthread_mutex_t mutex;
pthread_cond_t cond_var;

pthread_mutex_init(&mutex,NULL);
pthread_cond_init(&cond_var,NULL);
```

POSIX Condition Variables

- Thread waiting for the condition **a == b** to become true:

```
pthread_mutex_lock(&mutex);
while (a != b)
     pthread_cond_wait(&cond_var, &mutex);

pthread_mutex_unlock(&mutex);
```

- Thread signaling another thread waiting on the condition variable:

```
pthread_mutex_lock(&mutex);
a = b;
pthread_cond_signal(&cond_var);
pthread_mutex_unlock(&mutex);
```



- Questions?

Acknowledgment:

This presentation is based on the companion materials provided with the book

Operating System Concepts – 10h Edition, by Silberschatz, Galvin and Gagne